宠物寄养模块讲解文档

一、数据库与实体类设计

1.1 核心实体表设计

1.1.1 寄养表(boarding)

字段名	数据类型	描述	约束条件
id	BIGINT	寄养ID	主键，自增
user_id	BIGINT	用户ID	外键关联user表
pet_name	VARCHAR(50)	宠物名称	非空
pet_type	VARCHAR(50)	宠物类型	非空
start_time	DATETIME	开始时间	非空
end_time	DATETIME	结束时间	非空
status	VARCHAR(20)	状态(已申请/已接受/进行中/已完成/已取消)	非空
requirements	TEXT	特殊要求	可选
price	DECIMAL(10,2)	价格	非空
create_time	DATETIME	创建时间	自动生成
update_time	DATETIME	更新时间	自动更新

Boarding类：对应寄养表，包含寄养基本信息
【springboot/src/main/java/org/example/springboot/entity/Boarding.java】

```
@Data
@TableName("boarding")
public class Boarding {
    /**
     * 寄养ID
```

半夜撕代码版权所有

```java
     */
    @TableId(value = "id", type = IdType.AUTO)
    private Long id;

    /**
     * 用户ID
     */
    private Long userId;

    /**
     * 宠物名称
     */
    private String petName;

    /**
     * 宠物类型
     */
    private String petType;

    /**
     * 开始时间
     */
    private LocalDateTime startTime;

    /**
     * 结束时间
     */
    private LocalDateTime endTime;

    /**
     * 状态(已申请/已接受/进行中/已完成/已取消)
     */
    private String status;

    // 其他属性...
}
```

1.2 核心属性说明

1. **寄养状态流转**：寄养申请有五种状态，分别是"已申请"、"已接受"、"进行中"、"已完成"和"已取消"，反映了寄养服务的完整生命周期
2. **价格计算**：根据宠物类型和寄养时长计算价格，不同宠物类型有不同的价格标准
3. **特殊要求**：用户可以在requirements字段中提出特殊照顾要求，如饮食偏好、医疗需求等
4. **用户关联**：通过userId字段关联用户表，实现用户与寄养申请的一对多关系

半夜撕代码版权所有

二、各功能详细讲解

2.1 寄养申请功能

2.1.1 功能概述

寄养申请功能允许用户为自己的宠物申请临时寄养服务。用户需要填写宠物基本信息、寄养时间段和特殊要求等，系统会根据宠物类型和寄养时长自动计算价格，并将申请提交给管理员审核。

2.1.2 详细讲解功能实现流程

用户访问寄养申请页面时，前端通过BoardingApply.vue组件初始化，展示寄养服务介绍和申请表单。用户填写宠物名称、选择宠物类型、设定寄养时间段并提交特殊要求。系统会检查用户是否已登录，未登录则弹出登录提示对话框。

用户提交表单后，前端进行表单验证，确保必填字段已填写且格式正确，特别是检查结束时间必须晚于开始时间。验证通过后，通过request.post('/boarding')将数据提交到后端。

后端BoardingController接收请求，调用boardingService.addBoarding方法处理申请。该方法首先验证数据合法性，然后设置创建时间和初始状态为"已申请"，最后保存到数据库。申请成功后，前端会显示成功提示并引导用户查看自己的寄养记录。

2.1.3 关键代码讲解

1. 前端寄养申请表单
   【vue3/src/views/frontend/pet/BoardingApply.vue】

```
const boardingForm = reactive({
  petName: '',
  petType: '',
  startTime: '',
  endTime: '',
  requirements: '',
  status: '已申请',
  price: 0,
  userId: userStore.userInfo?.id || null
})

const submitForm = () => {
  formRef.value.validate(async (valid) => {
    if (valid) {
      if (!userStore.isLoggedIn) {
        loginDialogVisible.value = true
        return
      }

      submitting.value = true
      try {
```

半夜撕代码版权所有

```
        await request.post('/boarding', boardingForm, {
          successMsg: t('boarding.submitSuccess'),
          onSuccess: () => {
            router.push('/my-boarding')
          }
        })
      } catch (error) {
        console.error('提交寄养申请失败:', error)
      } finally {
        submitting.value = false
      }
    }
  })
}
```

这段代码实现了寄养申请表单的提交逻辑。首先验证表单数据，然后检查用户是否已登录，最后将数据提交到后端并在成功后跳转到"我的寄养"页面。

2. **后端寄养申请处理**
   【springboot/src/main/java/org/example/springboot/service/BoardingService.java】

```
@Transactional
public Boarding addBoarding(Boarding boarding) {
    // 校验数据
    validateBoardingData(boarding);

    // 设置创建时间和更新时间
    LocalDateTime now = LocalDateTime.now();
    boarding.setCreateTime(now);
    boarding.setUpdateTime(now);

    // 插入数据
    boardingMapper.insert(boarding);

    return boarding;
}

private void validateBoardingData(Boarding boarding) {
    // 校验宠物名称
    if (StringUtils.isBlank(boarding.getPetName())) {
        throw new ServiceException("宠物名称不能为空");
    }

    // 校验宠物类型
    if (StringUtils.isBlank(boarding.getPetType())) {
        throw new ServiceException("宠物类型不能为空");
    }
```

半夜撕代码版权所有

```
        // 校验开始时间
        if (boarding.getStartTime() == null) {
            throw new ServiceException("开始时间不能为空");
        }

        // 校验结束时间
        if (boarding.getEndTime() == null) {
            throw new ServiceException("结束时间不能为空");
        }

        // 校验结束时间必须大于开始时间
        if (boarding.getEndTime().isBefore(boarding.getStartTime())) {
            throw new ServiceException("结束时间必须大于开始时间");
        }

        // 校验状态
        if (StringUtils.isBlank(boarding.getStatus())) {
            throw new ServiceException("状态不能为空");
        }

        // 检查状态值是否合法
        if (!isValidStatus(boarding.getStatus())) {
            throw new ServiceException("无效的状态值");
        }
    }
```

这段代码实现了寄养申请的后端处理逻辑。首先通过validateBoardingData方法验证数据的完整性和正确性，包括检查必填字段、时间逻辑和状态值合法性，然后设置创建时间和更新时间，最后将数据插入数据库。

2.2 我的寄养记录功能

2.2.1 功能概述

我的寄养记录功能允许用户查看自己的所有寄养申请及其状态。用户可以按状态筛选记录，查看详细信息，并对"已申请"状态的申请进行取消操作。该功能提供了寄养服务全生命周期的可视化管理。

2.2.2 详细讲解功能实现流程

用户进入"我的寄养"页面时，MyBoarding.vue组件初始化并调用fetchBoardings方法。该方法通过request.get('/boarding/my')向后端发送请求，传递用户ID和可选的状态筛选条件。

后端BoardingController接收请求，调用boardingService.getBoardingsByUserId方法查询该用户的寄养记录。查询结果按创建时间倒序排列，并进行分页处理，然后返回给前端。

半夜撕代码版权所有

前端接收数据后，将寄养记录渲染为表格形式，显示宠物信息、寄养时间、状态和价格等。用户可以点击"详情"按钮查看完整信息，或对"已申请"状态的记录点击"取消"按钮进行取消操作。取消操作会调用request.put(`/boarding/${id}/cancel`)发送请求，后端更新状态为"已取消"。

2.2.3 关键代码讲解

1. **前端寄养记录获取**
   【vue3/src/views/frontend/pet/MyBoarding.vue】

```
const fetchBoardings = async () => {
  if (!userStore.isLoggedIn) {
    ElMessage.warning(t('boarding.needLogin'))
    router.push('/login')
    return
  }

  loading.value = true
  loadError.value = false

  try {
    await request.get('/boarding/my', {
      userId: userStore.userInfo.id,
      status: searchForm.status,
      currentPage: currentPage.value,
      size: pageSize
    }, {
      onSuccess: (res) => {
        boardingList.value = res.records
        total.value = res.total
      },
      onError: () => {
        loadError.value = true
      }
    })
  } catch (error) {
    console.error('获取寄养记录失败:', error)
    loadError.value = true
  } finally {
    loading.value = false
  }
}
```

这段代码实现了获取用户寄养记录的逻辑。首先检查用户登录状态，然后发送请求获取数据，包含用户ID、状态筛选和分页参数，成功后更新记录列表和总数。

半夜撕代码版权所有

2. 后端用户寄养记录查询
【springboot/src/main/java/org/example/springboot/service/BoardingService.java】

```
public Page<Boarding> getBoardingsByUserId(Long userId, String status,
Integer currentPage, Integer size) {
    LambdaQueryWrapper<Boarding> queryWrapper = new
LambdaQueryWrapper<>();

    // 添加用户ID条件
    queryWrapper.eq(Boarding::getUserId, userId);

    // 添加状态条件
    if (StringUtils.isNotBlank(status)) {
        queryWrapper.eq(Boarding::getStatus, status);
    }

    // 按创建时间倒序排序
    queryWrapper.orderByDesc(Boarding::getCreateTime);

    // 执行分页查询
    return boardingMapper.selectPage(new Page<>(currentPage, size),
queryWrapper);
}
```

这段代码实现了根据用户ID查询寄养记录的后端逻辑。构建查询条件，包括用户ID和可选的状态筛选，按创建时间倒序排序，最后执行分页查询并返回结果。

2.3 寄养管理功能

2.3.1 功能概述

寄养管理功能是后台管理系统的一部分，允许管理员查看和处理所有用户的寄养申请。管理员可以按宠物名称、类型和状态筛选记录，查看详情，编辑信息，更新状态，以及删除记录。该功能实现了寄养服务的全流程管理。

2.3.2 详细讲解功能实现流程

管理员进入寄养管理页面时，BoardingManage.vue组件初始化并调用fetchBoardings方法。该方法通过request.get('/boarding/page')向后端发送请求，传递筛选条件和分页参数。

后端BoardingController接收请求，调用boardingService.getBoardingsByPage方法查询寄养记录。查询结果会填充用户名信息，按创建时间倒序排列，并进行分页处理，然后返回给前端。

前端接收数据后，将寄养记录渲染为表格形式，显示ID、宠物信息、时间、状态、价格和用户等。管理员可以对记录进行多种操作：

- 编辑：修改寄养信息，如宠物名称、价格等

- 删除：从系统中删除寄养记录
- 状态更新：根据当前状态提供不同的操作按钮，如接受申请、开始寄养、完成寄养等

当管理员更新寄养状态时，前端调用request.put(`/boarding/${id}/status`)发送请求，后端更新状态并返回更新后的记录。

2.3.3 关键代码讲解

1. **前端寄养状态更新**
   【vue3/src/views/backend/pet/BoardingManage.vue】

```
const handleUpdateStatus = async (row, newStatus) => {
  try {
    await ElMessageBox.confirm(
      t('backend.boarding.confirm.updateStatus', { status:
t(`backend.boarding.status.${newStatus}`) }),
      t('common.warning'),
      {
        confirmButtonText: t('common.confirm'),
        cancelButtonText: t('common.cancel'),
        type: 'warning'
      }
    )

    await request.put(`/boarding/${row.id}/status`, null, {
      status: newStatus
    }, {
      successMsg: t('backend.boarding.success.statusUpdated'),
      onSuccess: () => {
        handleRefresh()
      }
    })
  } catch (error) {
    if (error !== 'cancel') {
      console.error('更新寄养状态失败:', error)
    }
  }
}
```

这段代码实现了寄养状态更新的前端逻辑。首先弹出确认对话框，然后调用接口更新状态，成功后刷新列表。不同状态对应不同的操作按钮，实现了寄养服务的流程管理。

2. **后端寄养状态更新**
   【springboot/src/main/java/org/example/springboot/service/BoardingService.java】

```
@Transactional
public Boarding updateBoardingStatus(Long id, String status) {
```

半夜撕代码版权所有

```
        // 校验状态
        if (StringUtils.isBlank(status)) {
            throw new ServiceException("状态不能为空");
        }

        // 检查状态值是否合法
        if (!isValidStatus(status)) {
            throw new ServiceException("无效的状态值");
        }

        // 校验寄养信息是否存在
        Boarding boarding = boardingMapper.selectById(id);
        if (boarding == null) {
            throw new ServiceException("寄养信息不存在");
        }

        // 更新状态
        boarding.setStatus(status);
        boarding.setUpdateTime(LocalDateTime.now());
        boardingMapper.updateById(boarding);

        return boarding;
    }
```

这段代码实现了寄养状态更新的后端逻辑。首先验证状态值的合法性，然后检查寄养记录是否存在，最后更新状态和更新时间，并保存到数据库。

三、总结

3.1 寄养申请功能实现

寄养申请功能通过前端BoardingApply.vue组件和后端BoardingController实现。用户填写宠物信息和寄养时间后提交申请，前端进行表单验证，确保数据完整性和时间逻辑正确性。后端进一步验证数据，设置初始状态为"已申请"，并保存到数据库。整个流程设计合理，提供了良好的用户体验，同时确保了数据的安全性和一致性。

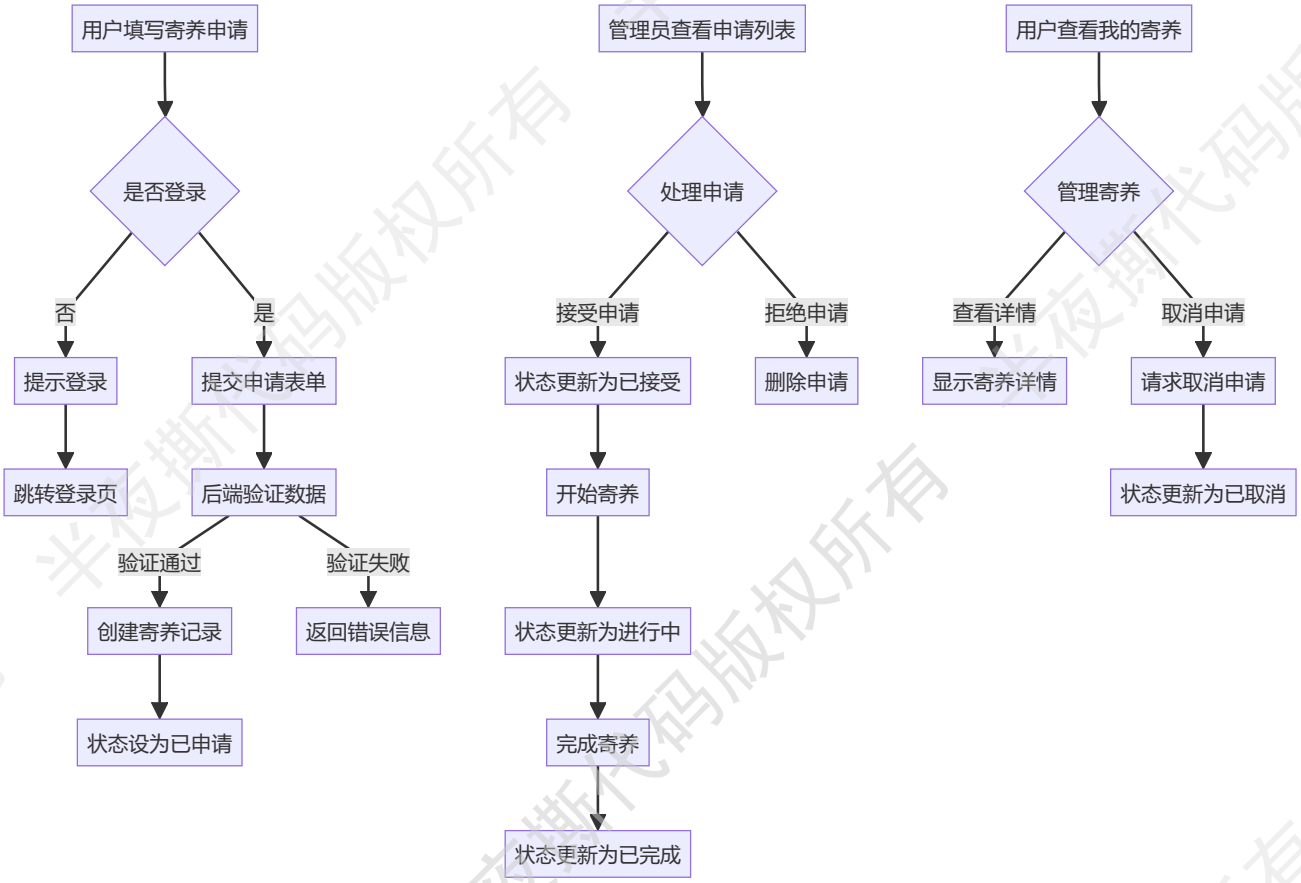
3.2 我的寄养记录功能实现

我的寄养记录功能通过MyBoarding.vue组件展示用户的寄养申请历史。用户可以按状态筛选记录，查看详情，并对"已申请"状态的记录进行取消操作。后端通过userId条件查询，确保用户只能查看自己的记录。界面设计清晰，状态使用不同颜色的标签区分，方便用户快速了解申请进展。

半夜撕代码版权所有

3.3 寄养管理功能实现

寄养管理功能通过BoardingManage.vue组件为管理员提供全面的寄养服务管理能力。管理员可以查看所有用户的寄养申请，进行筛选和分页，编辑信息，更新状态，以及删除记录。特别是状态流转功能，通过不同的操作按钮引导管理员按照"已申请→已接受→进行中→已完成"的流程处理寄养服务，确保服务质量和用户满意度。



半夜撕代码版权所有